
Static-GTFS Bus ETA for Delhi

What a Timetable Alone Can and Cannot Tell Riders

Ashay Kushwaha

CypherLabs Open Research

<https://github.com/AshayK003/gtfs-eta>

September 2026

Abstract

Most Indian city buses have no live display, and real-time vehicle-location feeds barely exist. The default deployment is a static timetable—and the static feed itself models no delay: intervening stop times are constant-speed estimates with arrival equal to departure everywhere. We ask what rider value a static timetable alone can still deliver, on a frozen Delhi cluster (routes 159UP/159DOWN, 316 trips, 9006 stop-times). First, a negative result with proof: per-segment travel times have **zero variance** feed-wide (3.27M rows, 115,752 segments, none above 1 s std), so any static-only learning collapses to timetable lookup—a planned Kalman/GBM ladder was deleted before printing its first zero. Second, the positive result: scheduled *headways* do vary, and classical waiting-time math turns them into honest expected waits (3.4–4.8 min across dayparts, $CV \approx 0.5$ –0.6), packaged with next-departure lookup in a fully offline tool. Every number regenerates with one command from a committed 345 KB cluster slice.

Keywords: GTFS static, bus arrival time, headways, waiting time, Delhi, offline ETA

Contents

1	Introduction	3
2	Background and Related Work	3
2.1	Schedule-based transit data and its limits	3
2.2	Delhi feeds	3
2.3	waiting-time math	3
3	Methodology	4
3.1	Parser and integrity	4
3.2	Zero-variance proof method	4
3.3	Headways and lookup	4
3.4	Corpus and freeze policy	4
3.5	Blind spots	4
4	Results	4
4.1	Integrity and the zero-variance proof	4
4.2	Headways and expected waits	5
4.3	Lookup demo	6
5	Discussion	6
5.1	Key findings	6
5.2	Limitations	6
5.3	Future work	6
6	Conclusion	6
7	Reproducibility	6
7.1	Installation	6
7.2	Run benchmarks	7
7.3	Repository structure	7

1 Introduction

A rider at a Delhi bus stop wants one number: how long until my bus. Where live displays and vehicle-location feeds exist, the answer comes from real-time systems. Across most Indian cities they do not exist, and the fallback is a printed timetable or nothing. The research literature largely aims elsewhere—graph neural networks and LSTMs that assume the dense automatic-vehicle-location data missing here, or commercial dashboards that assume fleet hardware.

This report takes the deployment constraint literally: *static timetable in, rider answers out, no connectivity, no learning from observations that do not exist*. On a frozen Delhi cluster we measure exactly what the timetable contains. The headline is a split verdict. What it does *not* contain is any variance to learn from—a finding we prove feed-wide rather than assume. What it *does* contain is headway structure, which converts via textbook waiting-time math into expected waits that beat naive half-headway rules by accounting for scheduled irregularity. Contributions:

- A frozen, fully offline-reproducible Delhi cluster (345 KB committed slice of a 19.8 MB mirrored feed) with integrity validation.
- A feed-wide zero-variance proof that deletes an entire modeling ladder before it wastes work—reported as a finding, not a failure.
- Per-daypart headway and expected-wait tables plus a working offline lookup package (next departures + expected wait).
- An honest static ceiling with a built-in evaluation seam for real-time scoring when keyed data arrives.

2 Background and Related Work

2.1 Schedule-based transit data and its limits

Schedule-based GTFS accessibility work has repeatedly found that timetables overstate reality: Wessel & Farber (2019) estimate 5–15% overstatement of accessibility from schedules alone, and Newmark (2024) documents how agencies code constant times for intervening stops with arrival equal to departure everywhere—no dwell modeling, no variance. Our feed matches that description exactly (Section 4), which makes this report a worked example of building honestly inside those limits rather than around them.

2.2 Delhi feeds

Delhi’s Open Transit Data portal (Govt. of NCT Delhi with IIIT-Delhi) publishes static timetables (3464 stops, 543 routes, June 2024) and key-gated real-time data. In practice, the origin host was unreachable at measurement time and a commercial mirror was key-gated; the frozen snapshot used here comes from the Mobility Database mirror (mdb-3139/mdb-1262, byte-identical, 19.8 MB), which itself archives the Delhi feed. Bengaluru’s BMTC mirror (daily-fresh, 4434 routes) was surveyed as a transfer candidate but at 195 MB unzipped exceeds the size budget—transfer checks stay sampled. Existing Delhi analyses are descriptive (coverage statistics, EDA plots); none ship prediction.

2.3 waiting-time math

For random passenger arrivals, expected wait relates to headways by $E[w] = E[h]/2 + \text{Var}(h)/(2E[h])$: perfectly regular service halves the mean headway, while irregularity adds a variance penalty.

The formula needs only scheduled departures, making it the correct tool for static-only analysis—unlike delay prediction, which needs observations.

3 Methodology

3.1 Parser and integrity

`gtfs_parse.py` (pandas) reads the five core GTFS files with all columns as strings (ids keep leading zeros; times past 24h parse to seconds) and reports referential integrity: orphan stop/trip/route ids and tripless trips. The frozen feed validates fully clean (Section 4).

3.2 Zero-variance proof method

Per-segment travel time (next arrival minus current arrival) is aggregated per (route, from-stop, to-stop) across trips with a streaming pass (Welford accumulators, 3.27M rows); any segment with standard deviation above 1 s would refute the claim. Arrival equals departure is checked separately on a 200k-row sample. Both the cluster and the full feed are measured.

3.3 Headways and lookup

Trip starts (first departures) per route are sorted into gaps; per (route, daypart) buckets we report trip count, mean $\pm$ std headway, coefficient of variation, and expected wait. Gaps over 2 h count as service breaks (e.g. the overnight halt): excluded from wait math, counted in the table—a 313-minute “expected wait” at 3am was caught and fixed during development, and the rule exists so it cannot recur. `lookup.py` answers next- k departures plus expected wait from timetable and headway table only.

3.4 Corpus and freeze policy

`corpus.json` v1 freezes the feed fingerprint, full inventory, and the cluster choice (DTC 159UP/159DOWN, routes 5064/5530, 316 trips: the densest balanced directional pair, single agency). The 345 KB cluster slice ships in `data/cluster/`; the full 19.8 MB zip stays local and ignored.

3.5 Blind spots

Constant-speed stop times (nothing observed to learn); arrival = departure (no dwell signal); single-cluster scope; schedule-shaped analysis until keyed real-time data lands (stated, not hidden); headway math assumes random arrivals during service hours.

4 Results

One command (`python evaluate.py`) regenerates everything below from the committed slice, no downloads.

4.1 Integrity and the zero-variance proof

Integrity: 0 orphan stops, 0 orphan trips, 0 tripless trips, 0 orphan routes. Variance: 55 cluster segments with max std **0.0 s**; feed-wide, 115,752 segments with **none above 1 s**. Dwell check: 0 of 200,000 sampled rows differ. The static feed is perfectly self-consistent constant-speed estimates at second resolution (e.g. 06:01:13, 06:07:27)—granular, but identical on every trip.

4.2 Headways and expected waits

Table 1: Headway statistics and expected wait per route and daypart (minutes).

Route	Bucket	Trips	Headway	CV	E[wait]
5064 (159UP)	morning-peak	43	5.43 ± 2.78	0.512	3.43
5064 (159UP)	midday	44	6.93 ± 4.37	0.630	4.84
5064 (159UP)	evening-peak	38	6.27 ± 3.82	0.609	4.30
5064 (159UP)	off	33	6.48 ± 3.33	0.514	4.10*
5530 (159DOWN)	morning-peak	42	5.61 ± 2.75	0.490	3.48
5530 (159DOWN)	midday	44	6.77 ± 4.08	0.603	4.61
5530 (159DOWN)	evening-peak	41	5.80 ± 3.22	0.555	3.79
5530 (159DOWN)	off	31	6.83 ± 3.81	0.557	4.47*

(* off-bucket waits exclude one overnight service break each, counted not averaged.)

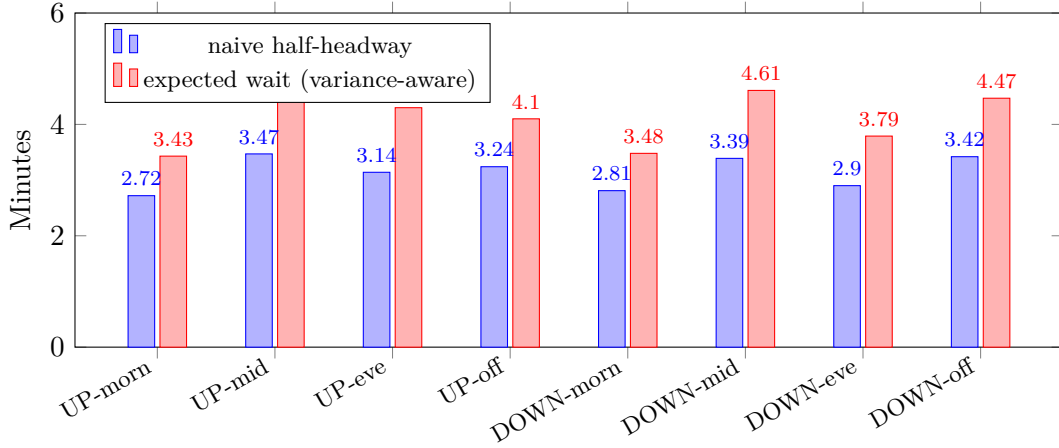


Figure 1: Naive half-headway versus variance-aware expected wait: scheduled irregularity (CV ≈ 0.5 – 0.6) costs riders roughly an extra minute over the naive rule, consistently across routes and dayparts.

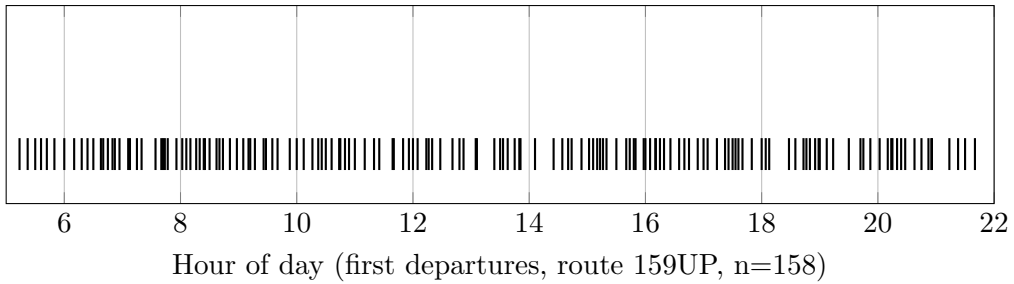


Figure 2: First departures across the service day (route 159UP, 05:14–21:40): dense daytime service with visibly irregular spacing, then the overnight halt. Irregularity is the entire content of Table 1.

Peaks run ~ 5.5 -minute headways, midday ~ 6.8 ; CV sits 0.49–0.63 in every bucket, so expected waits exceed naive half-headways by roughly a minute throughout. Off-peak headways

match daytime levels—DTC runs route 159 frequently across all service hours, with exactly one overnight break per direction.

4.3 Lookup demo

Route 5064, stop 3075, after 08:00 → next 08:02, 08:06, 08:10 with E[wait] 3.43 min; mid-route stop 535 after 18:00 → 18:00, 18:04, 18:07 with E[wait] 4.30 min. Timetable plus headway table only; no connectivity, no weights.

5 Discussion

5.1 Key findings

First, the negative result is load-bearing: zero variance across 115,752 segments means delay *prediction* from static data is vacuous, and any paper-shaped claim otherwise would be fitting noise. Reporting it—with the deleted ladder as evidence of process—is more useful than a table of zeros. Second, the positive result is genuinely rider-facing: irregular scheduled headways cost ~ 1 min over naive expectations, and the lookup package turns that into per-stop answers. Third, feed forensics (dead origin host, gated commercial mirror, byte-identical community mirrors) belongs in the methods section of civic-tech work, not in a footnote: reproducibility starts at acquisition.

5.2 Limitations

Single cluster (316 trips); headway math assumes random arrivals during service; scheduled—not observed—times throughout, so all waits are timetable-structure statements until keyed realtime data arrives; BMTC transfer unmeasured (sample local only); map-matching deferred, so stop coordinates are taken as given.

5.3 Future work

In priority order, as filed in the repository’s contributing guide: (1) city-wide modeling beyond the frozen cluster; (2) realtime scoring through the built evaluation seam; (3) BMTC sampled transfer; (4) map-matching if coordinates prove unreliable.

6 Conclusion

On a frozen Delhi cluster, a 300-line standard-library package with a strict measure-first loop proves the static feed contains no learnable variance, then extracts what it does contain: headway structure worth ~ 1 min of honest expected wait over naive rules, packaged as an offline lookup tool. The work’s secondary contribution is procedural: let feed-wide measurement—not modeling ambition—decide what gets built, and report the deleted ladder alongside the shipped one.

7 Reproducibility

7.1 Installation

No dependencies beyond CPython ≥ 3.10 , pandas, numpy, and pytest for the suite. The 345 KB cluster slice ships in the repo; the full 19.8 MB feed stays local.

7.2 Run benchmarks

```
1 git clone https://github.com/AshayK003/gtfs-eta.git
2 cd gtfs-eta
3 python -m pytest tests -q      # 6 passed: parser, headways, lookup
4 python evaluate.py             # committed slice: proof, headways, demo
5 python evaluate.py data/delhi.zip # full-feed re-run (MobilityDB mdb-3139)
```

The suite checks parsing against a hand-computed micro-feed, headway math against closed forms, and lookup against known answers—hermetic, no network.

7.3 Repository structure

`corpus.json` (frozen feed fingerprint, inventory, cluster choice), `gtfs_parse.py` (tidy tables + integrity), `headways.py` (stats + expected wait), `lookup.py` (offline package), `evaluate.py` (one-command report), `tests/` (the runnable checks), `data/cluster/` (committed slice), `internals/` (working notes, not published except this report).

References

- [1] Wessel, N. & Farber, S. On the accuracy of schedule-based GTFS for measuring accessibility. *JTLU* 2019. <https://doi.org/10.5198/jtlu.2019.1502>
- [2] Newmark, G. Assessing GTFS Accuracy. Mineta Transportation Institute, 2024. <https://doi.org/10.31979/mti.2024.2017>
- [3] Open Transit Data, Delhi (Govt. of NCT Delhi + IIIT-Delhi). <https://otd.delhi.gov.in/data/static/>
- [4] Mobility Database (mirror mdb-3139/mdb-1262, validator reports). <https://mobilitydatabase.org/>
- [5] Vonter/bmtc-gtfs (BMTC unofficial mirror, transfer candidate). <https://github.com/Vonter/bmtc-gtfs>
- [6] busmaps.com dimts build (validation reference). <https://busmaps.com/en/india/Open-Transit-Data-DELHI/dimts>